

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г. Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №4
по дисциплине: ТАиФЯ
по теме: Нисходящая обработка контекстно-свободных языков

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: Рязанов Ю. Д.

Цель работы: Изучить и научиться применять нисходящие методы обработки формальных языков.

Задания к работе:

1. Преобразовать исходную КС-грамматику в LL(1)-грамматику (см. варианты заданий).
 2. Определить множества ПЕРВЫХ для каждого символа LL(1)-грамматики.
 3. Определить множества СЛЕДУЮЩИХ для каждого символа LL(1)-грамматики.
 4. Определить множество ВЫБОРА для каждого правила LL(1)-грамматики.
 5. Написать программу-распознаватель методом рекурсивного спуска. Программа должна выводить последовательность номеров правил, применяемых при левом выводе обрабатываемой цепочки.
 6. Сформировать наборы тестовых данных. Тестовые данные должны содержать цепочки, принадлежащие языку, заданному грамматикой, (допустимые цепочки) и цепочки, не принадлежащие языку. Для каждой допустимой цепочки построить дерево вывода и левый вывод.
- Каждое правило грамматики должно использоваться в выводах допустимых цепочек хотя бы один раз.
7. Обработать цепочки из набора тестовых данных (см. п.6) программой-распознавателем.
 8. Построить нисходящий МП-распознаватель по LL(1)-грамматике.
 9. Написать программу-распознаватель, реализующую построенный нисходящий МП-распознаватель. Программа должна выводить на каждом шаге номер применяемого правила и промежуточную цепочку левого вывода.
 10. Обработать цепочки из набора тестовых данных (см. п.6) программой-распознавателем.

Вариант №9

1. $S \rightarrow OS$
2. $S \rightarrow O$
3. $O \rightarrow Y[S]$
4. $O \rightarrow \{Y[S]\}$
5. $O \rightarrow a = E$
6. $Y \rightarrow a = a$
7. $Y \rightarrow a < a$
8. $Y \rightarrow !(Y)$
9. $E \rightarrow (E + E)$
10. $E \rightarrow (E * E)$
11. $E \rightarrow -(E)$
12. $E \rightarrow a$

Выполнение работы:

1. Преобразовать исходную КС-грамматику в LL(1)-грамматику (см. варианты заданий).

1. $S \rightarrow OS$

2. $S \rightarrow O$

3. $O \rightarrow Y[S]$

4. $O \rightarrow \{Y[S]\}$

5. $O \rightarrow a = E$

6. $Y \rightarrow a = a$

7. $Y \rightarrow a < a$

8. $Y \rightarrow !(Y)$

9. $E \rightarrow (E + E)$

10. $E \rightarrow (E * E)$

11. $E \rightarrow -(E)$

12. $E \rightarrow a$

Преобразуем грамматику, выполнив левую факторизацию. Правило номер 3 выделено, потому что при раскрытии Y даст «а» в начале цепочки, что приведет к тому, что для правил 3 и 5 с одинаковой левой частью в начале правой части стоит один и тот же символ, что в конечном счете приведет к пересечению множеств ВЫБОРА для данных правил.

Подсветим правила, которые были изменены/добавлены.

1. $S \rightarrow OS'$

2. $S' \rightarrow OS'$

3. $S' \rightarrow \varepsilon$

4. $O \rightarrow aO'$

5. $O \rightarrow \{Y[S]\}$

6. $O \rightarrow !(Y)[S]$

7. $O' \rightarrow = E$

8. $O' \rightarrow Y'[S]$

9. $Y \rightarrow !(Y)$

10. $Y \rightarrow aY'$

11. $Y' \rightarrow = a$

12. $Y' \rightarrow < a$

13. $E \rightarrow -(E)$

14. $E \rightarrow a$

15. $E \rightarrow (EE'$

16. $E' \rightarrow * E)$

17. $E' \rightarrow +E)$

При раскрытии правила 8 мы вновь получим неоднозначность, ведь множество $\text{ВЫБОР}(O' \rightarrow Y'[S]) = \text{ПЕРВ}(Y') = \{< =\}$, что пересекается с $\text{ВЫБОР}(O' \rightarrow =E) = \{=\}$

Еще преобразуем грамматику.

1. $S \rightarrow OS'$
2. $S' \rightarrow OS'$
3. $S' \rightarrow \varepsilon$
4. $O \rightarrow aO'$
5. $O \rightarrow \{Y[S]\}$
6. $O \rightarrow !(Y)[S]$
7. $O' \rightarrow < a[S]$
8. $O' \rightarrow = R$
9. $R \rightarrow a[S]$
10. $R \rightarrow E$
11. $Y \rightarrow !(Y)$
12. $Y \rightarrow aY'$
13. $Y' \rightarrow = a$
14. $Y' \rightarrow < a$
15. $E \rightarrow -(E)$
16. $E \rightarrow a$
17. $E \rightarrow (EE'$
18. $E' \rightarrow * E)$
19. $E' \rightarrow +E)$

Правила 9 и 10 имеет аналогичную проблему, которую мы решали на предыдущем шаге – множества ВЫБОРА этих двух правил пересекаются по символу «а».

Преобразуем грамматику

1. $S \rightarrow OS'$
2. $S' \rightarrow OS'$
3. $S' \rightarrow \varepsilon$
4. $O \rightarrow aO'$
5. $O \rightarrow \{Y[S]\}$
6. $O \rightarrow !(Y)[S]$
7. $O' \rightarrow < a[S]$
8. $O' \rightarrow = R$
9. $R \rightarrow -(E)$
10. $R \rightarrow (EE'$
11. $R \rightarrow aT$
12. $T \rightarrow [S]$
13. $T \rightarrow \varepsilon$
14. $Y \rightarrow !(Y)$
15. $Y \rightarrow aY'$
16. $Y' \rightarrow = a$
17. $Y' \rightarrow < a$
18. $E \rightarrow -(E)$
19. $E \rightarrow a$
20. $E \rightarrow (EE'$
21. $E' \rightarrow * E)$
22. $E' \rightarrow +E)$

Готово, правил, которые бы имели пересекающиеся множества ВЫБОРА нет

2. Определить множества ПЕРВЫХ для каждого символа LL(1)-грамматики.

Выполним поэтапно:

	Множества ПЕРВЫХ
S	
S'	ε
O	a { !
O'	< =
R	- (a
T	ε [
Y	a !
Y'	= <
E	a – (
E'	+ *

В множество ПЕРВЫХ(S') и ПЕРВЫХ(S) добавим значения ПЕРВЫХ(O) по 1-му и 2-му правилу.

	Множества ПЕРВЫХ
S	a { !
S'	ε a { !
O	a { !
O'	< =
R	- (a
T	ε [
Y	a !
Y'	= <
E	a – (
E'	+ *

При дальнейшем рассмотрении множества ПЕРВЫХ не изменяются.

3. Определить множества СЛЕДУЮЩИХ для каждого символа LL(1)-грамматики.

Определим множества СЛЕДУЮЩИХ

	Множества ПЕРВЫХ	Множеств СЛЕДУЮЩИХ
S	$a \{ !$	$\neg]$
S'	$\varepsilon a \{ !$	$\neg]$
O	$a \{ !$	$a \{ ! \neg]$
O'	$< =$	$a \{ ! \neg]$
R	$- (a$	$a \{ ! \neg]$
T	$\varepsilon [$	$a \{ ! \neg]$
Y	$a !$	$[)$
Y'	$= <$	$[)$
E	$a - ($	$[) a \{ ! + *$
E'	$+ *$	$[) a \{ ! + *$

Мы определили множества СЛЕДУЮЩИХ.

4. Теперь определим множества ВЫБОРА для правил.

Грамматика	Множества ВЫБОР
1. $S \rightarrow OS'$	1. a { !
2. $S' \rightarrow OS'$	2. a { !
3. $S' \rightarrow \varepsilon$	3. $\neg$]
4. $O \rightarrow aO'$	4. a
5. $O \rightarrow \{Y[S]\}$	5. {
6. $O \rightarrow !(Y)[S]$	6. !
7. $O' \rightarrow < a[S]$	7. <
8. $O' \rightarrow = R$	8. =
9. $R \rightarrow -(E)$	9. -
10. $R \rightarrow (EE'$	10.(
11. $R \rightarrow aT$	11.a
12. $T \rightarrow [S]$	12.[
13. $T \rightarrow \varepsilon$	13.a { ! $\neg$]
14. $Y \rightarrow !(Y)$	14.!
15. $Y \rightarrow aY'$	15.a
16. $Y' \rightarrow = a$	16.=
17. $Y' \rightarrow < a$	17.<
18. $E \rightarrow -(E)$	18.-
19. $E \rightarrow a$	19.a
20. $E \rightarrow (EE'$	20.(
21. $E' \rightarrow * E)$	21.*
22. $E' \rightarrow +E)$	22.+

У всех правил с одинаковой левой частью множества ВЫБОР попарно не пересекаются. Следовательно, мы имеем LL(1) грамматику.

5. Написать программу-распознаватель методом рекурсивного спуска. Программа должна выводить последовательность номеров правил, применяемых при левом выводе обрабатываемой цепочки.

```
class Tree:
    def __init__(self, label="", rule_number=0, rule_string=""):
        self.children = []
        self.label = label
        self.rule_number = rule_number
        self.rule_string = rule_string

class Scan:
    def __init__(self):
        self.str = ""
        self.runes = []
        self.curr_pos = 0
        self.logger_scan = None # Placeholder for a logging instance
        self.logger_error = None # Placeholder for a logging instance
        self.tree_view = None
        self.is_error = False
        self.error = None

    def next_sym(self):
        if self.curr_pos >= self.size_runes():
            raise ValueError(
                f"current pos is <{self.curr_pos}> and length of string is <{self.size_runes()}>"
            )
        self.curr_pos += 1

    def size_runes(self):
        return len(self.runes)

    def curr_rune(self):
        if self.curr_pos >= self.size_runes():
            raise ValueError(
                f"current pos is <{self.curr_pos}> and length of string is <{self.size_runes()}>"
            )
        return self.runes[self.curr_pos]

    def is_curr_equal(self, r):
        try:
            curr_rune = self.curr_rune()
            if curr_rune == r:
                return True
            raise ValueError(f"current rune <{curr_rune}> is not equal to rune <{r}>.")
        except ValueError as e:
            raise ValueError(f"Error while comparing runes: {e}")

    def perform_step(self, r):
        if not self.is_curr_equal(r):
            raise ValueError(f"Mismatch at position {self.curr_pos} for rune <{r}>")
        node = Tree(label=str(r))
        self.next_sym()
        return node

    def gen_step(self, r):
        return lambda: self.perform_step(r)

    def throw_error(self, r):
        return lambda: (None, ValueError("wrong input"))
```

```

def init(self, input_str):
    self.str = input_str.strip()
    self.runes = list(self.str)
    self.is_error = False

def rule_record(self, parent, number, representation):
    if self.logger_scan:
        self.logger_scan.info(f"Applying rule number {number}: {representation}")
    parent.rule_number = number
    parent.rule_string = representation

def apply_steps(self, parent, steps):
    if self.is_error:
        return self.error

    for step in steps:
        try:
            node = step()
            if not node:
                node = Tree(label="ERROR", rule_number=0)
            parent.children.append(node)
        except Exception as e:
            self.is_error = True
            self.error = e
            parent.children.append(Tree(label="ERROR", rule_number=0))
            return e

    return None

def analyze(self):
    if self.logger_scan:
        self.logger_scan.info("\033[32mAnalysis started\033[0m")
    parent, err = self.rule_s()
    self.tree_view = parent
    if err and self.logger_error:
        self.logger_error.error(f"\033[31mAnalysis finished with error\033[0m: {err}")
    return err

def rule_s(self):
    parent = Tree(label="S")
    if self.curr_pos < self.size_runes() and any(
        self.is_curr_equal(c) for c in ['a', '{', '!']
    ):
        self.rule_record(parent, 1, "S -> OS'")
        self.apply_steps(parent, [self.rule_o, self.rule_s_])
    else:
        self.rule_record(parent, 1, "S -> OS'")
        self.apply_steps(parent, [self.throw_error('a')]) # force
    return parent, self.error

def is_end(self):
    return self.curr_pos >= self.size_runes()

def epsilon(self):
    return Tree(label="ε"), None

def rule_s_(self):
    parent = Tree(label="S'")
    if self.curr_pos < self.size_runes() and any(
        self.is_curr_equal(c) for c in ['a', '{', '!']
    ):
        self.rule_record(parent, 2, "S' -> OS'")
        self.apply_steps(parent, [self.rule_o, self.rule_s_])
    elif self.is_end() or self.is_curr_equal('}'):
        self.rule_record(parent, 3, "S' -> ε")
        self.apply_steps(parent, [self.epsilon])
    else:

```

```

        self.rule_record(parent, 2, "S' -> OS'")
        self.apply_steps(parent, [self.throw_error('a')])
    return parent, self.error

def rule_o(self):
    parent = Tree(label="O")
    if self.is_curr_equal('a'):
        self.rule_record(parent, 4, "O -> aO'")
        self.apply_steps(parent, [self.gen_step('a'), self.rule_o_])
    elif self.is_curr_equal('{'):
        self.rule_record(parent, 5, "O -> {Y[S]}")
        self.apply_steps(parent, [
            self.gen_step('{'),
            self.rule_y,
            self.gen_step('['),
            self.rule_s,
            self.gen_step(']'),
            self.gen_step('}')
        ])
    elif self.is_curr_equal('!'):
        self.rule_record(parent, 6, "O -> !(Y)[S]")
        self.apply_steps(parent, [
            self.gen_step('!'),
            self.gen_step('('),
            self.rule_y,
            self.gen_step(')'),
            self.gen_step('['),
            self.rule_s,
            self.gen_step(']')
        ])
    else:
        self.rule_record(parent, 4, "O -> aO'")
        self.apply_steps(parent, [self.throw_error('a')])
    return parent, self.error

def rule_o_(self):
    parent = Tree(label="O'")
    if self.is_curr_equal('<'):
        self.rule_record(parent, 7, "O' -> <a[S]")
        self.apply_steps(parent, [
            self.gen_step('<'),
            self.gen_step('a'),
            self.gen_step('['),
            self.rule_s,
            self.gen_step(']')
        ])
    elif self.is_curr_equal('='):
        self.rule_record(parent, 8, "O' -> =R")
        self.apply_steps(parent, [self.gen_step('='), self.rule_r])
    else:
        self.rule_record(parent, 7, "O' -> <a[S]")
        self.apply_steps(parent, [self.throw_error('<')])
    return parent, self.error

def rule_r(self):
    parent = Tree(label="R")
    if self.is_curr_equal('-'):
        self.rule_record(parent, 9, "R -> -(E)")
        self.apply_steps(parent, [
            self.gen_step('-'),
            self.gen_step('('),
            self.rule_e,
            self.gen_step(')')
        ])
    elif self.is_curr_equal('('):
        self.rule_record(parent, 10, "R -> (EE'")
        self.apply_steps(parent, [
            self.gen_step('('),

```

```

        self.rule_e,
        self.rule_e_
    ])
    elif self.is_curr_equal('a'):
        self.rule_record(parent, 11, "R -> aT")
        self.apply_steps(parent, [
            self.gen_step('a'),
            self.rule_t
        ])
    else:
        self.rule_record(parent, 9, "R -> -(E)")
        self.apply_steps(parent, [self.throw_error('-')])
    return parent, self.error

def rule_t(self):
    parent = Tree(label="T")
    if self.is_curr_equal('['):
        self.rule_record(parent, 12, "T -> [S]")
        self.apply_steps(parent, [
            self.gen_step('['),
            self.rule_s,
            self.gen_step(']')
        ])
    elif self.is_curr_equal('a') or self.is_curr_equal('{') or
self.is_curr_equal('}') or self.is_curr_equal(']') or self.is_end():
        self.rule_record(parent, 13, "T -> ε")
        self.apply_steps(parent, [self.epsilon])
    else:
        self.rule_record(parent, 12, "T -> [S]")
        self.apply_steps(parent, [self.throw_error('[')])
    return parent, self.error

def rule_y(self):
    parent = Tree(label="Y")
    if self.is_curr_equal('!'):
        self.rule_record(parent, 14, "Y -> !(Y)")
        self.apply_steps(parent, [
            self.gen_step('!'),
            self.gen_step('('),
            self.rule_y,
            self.gen_step(')')
        ])
    elif self.is_curr_equal('a'):
        self.rule_record(parent, 15, "Y -> aY'")
        self.apply_steps(parent, [
            self.gen_step('a'),
            self.rule_y_
        ])
    else:
        self.rule_record(parent, 14, "Y -> !(Y)")
        self.apply_steps(parent, [self.throw_error('!')])
    return parent, self.error

def rule_y_(self):
    parent = Tree(label="Y'")
    if self.is_curr_equal('='):
        self.rule_record(parent, 16, "Y' -> =a")
        self.apply_steps(parent, [
            self.gen_step('='),
            self.gen_step('a')
        ])
    elif self.is_curr_equal('<'):
        self.rule_record(parent, 17, "Y' -> <a")
        self.apply_steps(parent, [
            self.gen_step('<'),
            self.gen_step('a')
        ])
    else:

```

```

        self.rule_record(parent, 16, "Y' -> =a")
        self.apply_steps(parent, [self.throw_error('=')])
    return parent, self.error

def rule_e(self):
    parent = Tree(label="E")
    if self.is_curr_equal('('):
        self.rule_record(parent, 20, "E -> (EE'")
        self.apply_steps(parent, [
            self.gen_step('('),
            self.rule_e,
            self.rule_e_
        ])
    elif self.is_curr_equal('-'):
        self.rule_record(parent, 18, "E -> -(E)")
        self.apply_steps(parent, [
            self.gen_step('-'),
            self.gen_step('('),
            self.rule_e,
            self.gen_step(')')
        ])
    elif self.is_curr_equal('a'):
        self.rule_record(parent, 19, "E -> a")
        self.apply_steps(parent, [self.gen_step('a')])
    else:
        self.rule_record(parent, 19, "E -> a")
        self.apply_steps(parent, [self.throw_error('a')])
    return parent, self.error

def rule_e_(self):
    parent = Tree(label="E'")
    if self.is_curr_equal('*'):
        self.rule_record(parent, 21, "E' -> *E")
        self.apply_steps(parent, [
            self.gen_step('*'),
            self.rule_e,
            self.gen_step(')')
        ])
    elif self.is_curr_equal('+'):
        self.rule_record(parent, 22, "E' -> +E")
        self.apply_steps(parent, [
            self.gen_step('+'),
            self.rule_e,
            self.gen_step(')')
        ])
    else:
        self.rule_record(parent, 22, "E' -> +E")
        self.apply_steps(parent, [self.throw_error('+')])
    return parent, self.error

```

6. Сформируем тестовые данные. В задании сказано «Каждое правило грамматики должно использоваться в выводах допустимых цепочек хотя бы один раз.».

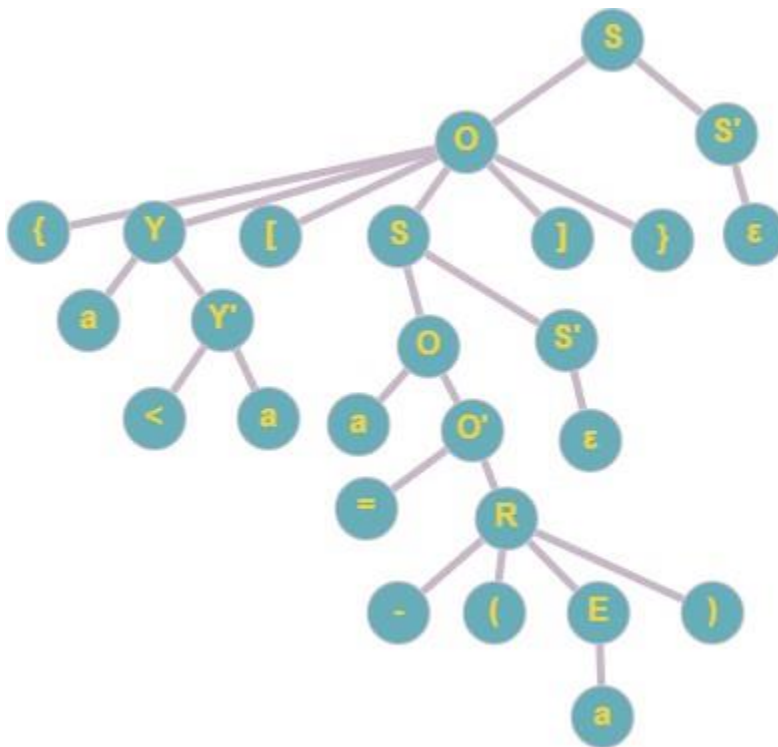
Сформируем такие наборы тестовых данных, чтобы охватить все правила грамматики.

1. Допустимая цепочка: $\{a < a[a = -(a)]\}$

Порядок применения правил: 1, 5, 15, 17, 1, 4, 8, 9, 19, 3, 3

Левый вывод: $S \Rightarrow OS' \Rightarrow \{Y[S]\}S' \Rightarrow \{aY'[S]\}S' \Rightarrow \{a < a[S]\}S' \Rightarrow \{a < a[OS']\}S' \Rightarrow \{a < a[aO'S']\}S' \Rightarrow \{a < a[a=RS']\}S' \Rightarrow \{a < a[a=-(E)S']\}S' \Rightarrow \{a < a[a=-(a)S']\}S' \Rightarrow \{a < a[a=-(a)]\}S' \Rightarrow \{a < a[a=-(a)]\}$

Дерево вывода:

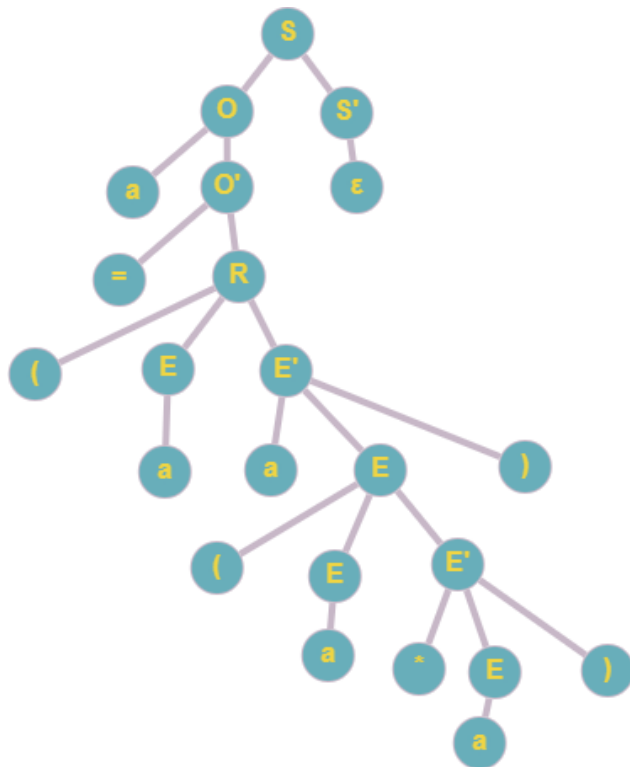


2. Допустимая цепочка: $a=(a+(a*a))$

Порядок применения правил: 1, 4, 8, 10, 19, 22, 20, 19, 21, 19, 3

Левый вывод: $S \Rightarrow OS' \Rightarrow aO'S' \Rightarrow a=RS' \Rightarrow a=(EE'S' \Rightarrow a=(aE'S' \Rightarrow a=(a+E)S' \Rightarrow a=(a+(EE')S' \Rightarrow a=(a+(aE')S' \Rightarrow a=(a+(a*E))S' \Rightarrow a=(a+(a*a))S' \Rightarrow a=(a+(a*a))$

Дерево вывода:



3. Допустимая цепочка: $!(!(a=a))[a=a[a=a]]a<a[a=(-(a)*a)]$

Порядок применения правил: 1, 6, 14, 15, 16, 1, 4, 8, 11, 12, 1, 4, 8, 11, 13, 3, 3, 2, 4, 7, 1, 4, 8, 10, 18, 19, 21, 19, 3, 3

Левый вывод: $S \Rightarrow OS' \Rightarrow !(Y)[S]S' \Rightarrow !(Y)[S]S' \Rightarrow !(aY)[S]S' \Rightarrow$
 $!(a=a)[S]S' \Rightarrow !(a=a)[OS']S' \Rightarrow !(a=a)[aO'S']S' \Rightarrow !(a=a)[a=RS']S' \Rightarrow$
 $!(a=a)[a=aTS']S' \Rightarrow !(a=a)[a=a[S]S']S' \Rightarrow !(a=a)[a=a[OS']S']S' \Rightarrow$
 $!(a=a)[a=a[aO'S']S']S' \Rightarrow !(a=a)[a=a[a=RS']S']S' \Rightarrow$
 $!(a=a)[a=a[a=aTS']S']S' \Rightarrow !(a=a)[a=a[a=aS']S']S' \Rightarrow$
 $!(a=a)[a=a[a=a]S']S' \Rightarrow !(a=a)[a=a[a=a]]S' \Rightarrow !(a=a)[a=a[a=a]]OS' \Rightarrow$
 $!(a=a)[a=a[a=a]]aO'S' \Rightarrow !(a=a)[a=a[a=a]]a<a[S]S' \Rightarrow$
 $!(a=a)[a=a[a=a]]a<a[OS']S' \Rightarrow !(a=a)[a=a[a=a]]a<a[aO'S']S' \Rightarrow$
 $!(a=a)[a=a[a=a]]a<a[a=RS']S' \Rightarrow !(a=a)[a=a[a=a]]a<a[a=(EE'S')]S' \Rightarrow$
 $!(a=a)[a=a[a=a]]a<a[a=(-(E)E'S')]S' \Rightarrow !(a=a)[a=a[a=a]]a<a[a=(-(a)E'S')]S'$
 $\Rightarrow !(a=a)[a=a[a=a]]a<a[a=(-(a)*E)S']S' \Rightarrow !(a=a)[a=a[a=a]]a<a[a=(-$
 $(a)*a)S']S' \Rightarrow !(a=a)[a=a[a=a]]a<a[a=(-(a)*a)]S' \Rightarrow$
 $!(a=a)[a=a[a=a]]a<a[a=(-(a)*a)]$

Дерево вывода:



Отметим, что три вышеописанных цепочки охватили все правила грамматики.

4. Недопустимая цепочка
Пустая цепочка
5. Недопустимая цепочка
 $a=(a+$
6. Недопустимая цепочка
 $\{a<a\}$

1.

```
Ввод: {a[a[a--(a)]]}
SCAN: 05:42:37 Начало анализа
SCAN: 05:42:37 Применяется правило номер 1: S -> OS'
SCAN: 05:42:37 Применяется правило номер 5: O -> {Y[S]}
SCAN: 05:42:37 Применяется правило номер 15: Y -> aY'
SCAN: 05:42:37 Применяется правило номер 17: Y' -> <a
SCAN: 05:42:37 Применяется правило номер 1: S -> OS'
SCAN: 05:42:37 Применяется правило номер 4: O -> aO'
SCAN: 05:42:37 Применяется правило номер 8: O' -> =R
SCAN: 05:42:37 Применяется правило номер 9: R -> -(E)
SCAN: 05:42:37 Применяется правило номер 19: E -> a
SCAN: 05:42:37 Применяется правило номер 3: S' -> ε
SCAN: 05:42:37 Применяется правило номер 3: S' -> ε

Левый вывод: S => OS' => {Y[S]}S' => {aY'[S]}S' => {a[a[S]]S'} => {a[a[OS']]S'} => {a[a[aO'S']]S'} => {a[a[a=RS']]S'} => {a[a[a--(E)S']]S'} => {a[a[a--(a)S']]S'} => {a[a[a--(a)]]}S' => {a[a[a--(a)]]}
```

2.

```
Ввод: a=(a+(a*a))
SCAN: 05:43:27 Начало анализа
SCAN: 05:43:27 Применяется правило номер 1: S -> OS'
SCAN: 05:43:27 Применяется правило номер 4: O -> aO'
SCAN: 05:43:27 Применяется правило номер 8: O' -> =R
SCAN: 05:43:27 Применяется правило номер 10: R -> (EE'
SCAN: 05:43:27 Применяется правило номер 19: E -> a
SCAN: 05:43:27 Применяется правило номер 22: E' -> +E)
SCAN: 05:43:27 Применяется правило номер 20: E -> (EE'
SCAN: 05:43:27 Применяется правило номер 19: E -> a
SCAN: 05:43:27 Применяется правило номер 21: E' -> *E)
SCAN: 05:43:27 Применяется правило номер 19: E -> a
SCAN: 05:43:27 Применяется правило номер 3: S' -> ε

Левый вывод: S => OS' => aO'S' => a=RS' => a=(EE'S' => a=(aE'S' => a=(a+E)S' => a=(a+(EE')S' => a=(a+(aE')S' => a=(a+(a*E))S' => a=(a+(a*a))S' => a=(a+(a*a))
```

3.

```

Ввод: 1(1(a-a))(a-a[a-a])a(a-(a)*a)
SCAN: 05:43:55 Начало анализа
SCAN: 05:43:55 Применяется правило номер 1: S -> OS'
SCAN: 05:43:55 Применяется правило номер 6: O -> 1(Y)[S]
SCAN: 05:43:55 Применяется правило номер 14: Y -> 1(Y)
SCAN: 05:43:55 Применяется правило номер 15: Y' -> aY'
SCAN: 05:43:55 Применяется правило номер 16: Y' -> -a
SCAN: 05:43:55 Применяется правило номер 1: S -> OS'
SCAN: 05:43:55 Применяется правило номер 4: O -> aO'
SCAN: 05:43:55 Применяется правило номер 8: O' -> -R
SCAN: 05:43:55 Применяется правило номер 11: R -> aT
SCAN: 05:43:55 Применяется правило номер 12: T -> [S]
SCAN: 05:43:55 Применяется правило номер 1: S -> OS'
SCAN: 05:43:55 Применяется правило номер 4: O -> aO'
SCAN: 05:43:55 Применяется правило номер 8: O' -> -R
SCAN: 05:43:55 Применяется правило номер 11: R -> aT
SCAN: 05:43:55 Применяется правило номер 13: T -> ε
SCAN: 05:43:55 Применяется правило номер 3: S' -> ε
SCAN: 05:43:55 Применяется правило номер 3: S' -> ε
SCAN: 05:43:55 Применяется правило номер 2: S' -> OS'
SCAN: 05:43:55 Применяется правило номер 4: O -> aO'
SCAN: 05:43:55 Применяется правило номер 7: O' -> <a[S]
SCAN: 05:43:55 Применяется правило номер 1: S -> OS'
SCAN: 05:43:55 Применяется правило номер 4: O -> aO'
SCAN: 05:43:55 Применяется правило номер 8: O' -> -R
SCAN: 05:43:55 Применяется правило номер 10: R -> (EE'
SCAN: 05:43:55 Применяется правило номер 18: E -> -(E)
SCAN: 05:43:55 Применяется правило номер 19: E -> a
SCAN: 05:43:55 Применяется правило номер 21: E' -> *E)
SCAN: 05:43:55 Применяется правило номер 19: E -> a
SCAN: 05:43:55 Применяется правило номер 3: S' -> ε
SCAN: 05:43:55 Применяется правило номер 3: S' -> ε

```

4.

```

Ввод:
SCAN: 05:50:16 Начало анализа

```

```

Левый вывод: S => ERROR

```

5.

```
Ввод: a=(a+
SCAN: 05:50:45 Начало анализа
SCAN: 05:50:45 Применяется правило номер 1:    S -> OS'
SCAN: 05:50:45 Применяется правило номер 4:    O -> aO'
SCAN: 05:50:45 Применяется правило номер 8:    O' -> =R
SCAN: 05:50:45 Применяется правило номер 10:   R -> (EE'
SCAN: 05:50:45 Применяется правило номер 19:   E -> a
SCAN: 05:50:45 Применяется правило номер 22:   E' -> +E)

Левый вывод: S => OS' => aO'S' => a=RS' => a=(EE'S' => a=(aE'S' => a=(a+E)S' => ERROR
```

6.

```
Ввод: {a<a]
SCAN: 05:51:07 Начало анализа
SCAN: 05:51:07 Применяется правило номер 1:    S -> OS'
SCAN: 05:51:07 Применяется правило номер 5:    O -> {Y[S]}
SCAN: 05:51:07 Применяется правило номер 15:   Y -> aY'
SCAN: 05:51:07 Применяется правило номер 17:   Y' -> <a

Левый вывод: S => OS' => {Y[S]}S' => {aY'[S]}S' => {a<a[S]}S' => ERROR
```

Все результаты совпали с ожидаемыми.

7. Построить нисходящий МП-распознаватель по LL(1)-грамматике.

Построим таблицу МП-распознавателя.

[illegible]

Н.с.м.: SΔ

1. ЗАМЕНИТЬ(S'O), держать
2. ЗАМЕНИТЬ(S'O), держать
3. Вытолкнуть, держать
4. ЗАМЕНИТЬ(O'), сдвиг
5. ЗАМЕНИТЬ{ }S[Y], сдвиг
6. ЗАМЕНИТЬ(S[])Y(), сдвиг
7. ЗАМЕНИТЬ(S[a], сдвиг
8. ЗАМЕНИТЬ(R), сдвиг
9. ЗАМЕНИТЬ()E(), сдвиг
10. ЗАМЕНИТЬ(E'E), сдвиг
11. ЗАМЕНИТЬ(T), сдвиг
12. ЗАМЕНИТЬ(S), сдвиг
13. Вытолкнуть, держать
14. ЗАМЕНИТЬ()Y(), сдвиг
15. ЗАМЕНИТЬ(Y'), сдвиг
16. ЗАМЕНИТЬ(a), сдвиг
17. ЗАМЕНИТЬ(a), сдвиг
18. ЗАМЕНИТЬ()E(), сдвиг
19. ЗАМЕНИТЬ(), сдвиг
20. ЗАМЕНИТЬ(E'E), сдвиг
21. ЗАМЕНИТЬ()E), сдвиг
22. ЗАМЕНИТЬ()E), сдвиг

8. Написать программу-распознаватель, реализующую построенный нисходящий МП-распознаватель. Программа должна выводить на каждом шаге номер применяемого правила и промежуточную цепочку левого вывода.

```
import logging

EndLineSymbol = '\n'
EndStackSymbol = 'EndStackSymbol'

class MP:
    def __init__(self):
        self.stack = []
        self.inner = "S"
        self.CurrPos = 0
        self.ScanLogger = None

    def init(self, ScanLogger):
        self.stack = ["S", EndStackSymbol]
        self.inner = "S"
        self.ScanLogger = ScanLogger

    def pop(self):
        if self.stack[0] == EndStackSymbol:
            raise ValueError("try of pop from the end of the stack")
        self.stack.pop(0)

    def replace(self, replacement):
        if self.stack[0] == EndStackSymbol:
            raise ValueError("try of replace the end of the stack")
        self.pop()
        self.stack = replacement + self.stack
        if self.stack[0] == "":
            self.pop()

    def analyze(self, input_str):
        str_input = input_str.strip() + EndLineSymbol
        runes = list(str_input)
        process = True

        self.ScanLogger.info(f"Промежуточное представление: \t{self.inner}")

        while process:
            stack_symbol = self.stack[0]
            str_symbol = runes[self.CurrPos]

            action = Table.get(stack_symbol, {}).get(str_symbol)

            if not action:
                raise ValueError(f"No rule for stack symbol {stack_symbol} and input symbol {str_symbol}")

            if action["Action"] == "ActionPop":
                self.pop()
            elif action["Action"] == "ActionReplace":
                self.replace(action["ReplaceString"])
            elif action["Action"] == "ActionDone":
                process = False
            else:
                raise ValueError(f"Unknown action: {action['Action']}")

            if "Move" in action and action["Move"]:
```

```

        self.CurrPos += 1

        self.ScanLogger.info(f"Применяется правило: {self.stack}, Промежуточное
представление: {str_input[self.CurrPos:]}")

def reverse(arr):
    arr.reverse()

def init_rules(rules):
    for rule in rules:
        reverse(rule["ReplaceString"])

Rules = [
    {"Move": True, "Action": "ActionReplace", "ReplaceString": ["R"], "RuleNum": 8},
    {"Move": True, "Action": "ActionReplace", "ReplaceString": [")", "E", "("],
"RuleNum": 9},
]

Table = {
    "S": {
        "{": Rules[0],
        "a": Rules[0],
        "!": Rules[0],
    },
}

init_rules(Rules)

logger = logging.getLogger("MPLogger")
logging.basicConfig(level=logging.INFO)

def main():
    import sys

    print("Ввод: ", end="")
    input_str = sys.stdin.readline().strip()

    mp = MP()
    mp.init(logger)

    try:
        mp.analyze(input_str)
    except Exception as e:
        print(f"MP SCAN: {e}")

if __name__ == "__main__":
    main()

```

9. Обработать цепочки из набора тестовых данных (см. п.б) программой-распознавателем.

Результаты работы реализованного МП-распознавателя:

1.

```
Ввод: {a<a[a=-(a)]}
MP SCAN: 05:55:11 Промежуточное представление: S
MP SCAN: 05:55:11 Применяется правило 1: S -> OS'
MP SCAN: 05:55:11 Промежуточное представление: OS'
MP SCAN: 05:55:11 Применяется правило 5: O -> {Y[S]}
MP SCAN: 05:55:11 Промежуточное представление: {Y[S]}S'
MP SCAN: 05:55:11 Применяется правило 15: Y -> aY'
MP SCAN: 05:55:11 Промежуточное представление: {aY'[S]}S'
MP SCAN: 05:55:11 Применяется правило 17: Y' -> <a
MP SCAN: 05:55:11 Промежуточное представление: {a<a[S]}S'
MP SCAN: 05:55:11 Применяется правило 1: S -> OS'
MP SCAN: 05:55:11 Промежуточное представление: {a<a[OS']}S'
MP SCAN: 05:55:11 Применяется правило 4: O -> aO'
MP SCAN: 05:55:11 Промежуточное представление: {a<a[aO'S']}S'
MP SCAN: 05:55:11 Применяется правило 8: O' -> =R
MP SCAN: 05:55:11 Промежуточное представление: {a<a[a=RS']}S'
MP SCAN: 05:55:11 Применяется правило 9: R -> -(E)
MP SCAN: 05:55:11 Промежуточное представление: {a<a[a=-(E)S']}S'
MP SCAN: 05:55:11 Применяется правило 19: E -> a
MP SCAN: 05:55:11 Промежуточное представление: {a<a[a=-(a)S']}S'
MP SCAN: 05:55:11 Применяется правило 3: S' -> ε
MP SCAN: 05:55:11 Промежуточное представление: {a<a[a=-(a)]}S'
MP SCAN: 05:55:11 Применяется правило 3: S' -> ε
MP SCAN: 05:55:11 Промежуточное представление: {a<a[a=-(a)]}
```

2.

```
Ввод: a=(a+(a*a))
MP SCAN: 05:55:39 Промежуточное представление: S
MP SCAN: 05:55:39 Применяется правило 1: S -> OS'
MP SCAN: 05:55:39 Промежуточное представление: OS'
MP SCAN: 05:55:39 Применяется правило 4: O -> aO'
MP SCAN: 05:55:39 Промежуточное представление: aO'S'
MP SCAN: 05:55:39 Применяется правило 8: O' -> =R
MP SCAN: 05:55:39 Промежуточное представление: a=RS'
MP SCAN: 05:55:39 Применяется правило 10: R -> (EE'
MP SCAN: 05:55:39 Промежуточное представление: a=(EE'S'
MP SCAN: 05:55:39 Применяется правило 19: E -> a
MP SCAN: 05:55:39 Промежуточное представление: a=(aE'S'
MP SCAN: 05:55:39 Применяется правило 22: E' -> +E)
MP SCAN: 05:55:39 Промежуточное представление: a=(a+E)S'
MP SCAN: 05:55:39 Применяется правило 20: E -> (EE'
MP SCAN: 05:55:39 Промежуточное представление: a=(a+(EE'))S'
MP SCAN: 05:55:39 Применяется правило 19: E -> a
MP SCAN: 05:55:39 Промежуточное представление: a=(a+(aE'))S'
MP SCAN: 05:55:39 Применяется правило 21: E' -> *E)
MP SCAN: 05:55:39 Промежуточное представление: a=(a+(a*E))S'
MP SCAN: 05:55:39 Применяется правило 19: E -> a
MP SCAN: 05:55:39 Промежуточное представление: a=(a+(a*a))S'
MP SCAN: 05:55:39 Применяется правило 3: S' -> ε
MP SCAN: 05:55:39 Промежуточное представление: a=(a+(a*a))
```

3.

[illegible]

4.

```
Ввод:
MP SCAN: 05:56:37 Промежуточное представление: S
MP SCAN: 05:56:37 Ошибка
MP SCAN: 05:56:37 Stack: SΔ
MP SCAN: 05:56:37 Оставшаяся часть строки: -\
error during analyze. stack symbol <S> and string symbol <->
```

5.

```
Ввод: a=(a+
MP SCAN: 05:56:58 Промежуточное представление: S
MP SCAN: 05:56:58 Применяется правило 1: S -> OS'
MP SCAN: 05:56:58 Промежуточное представление: OS'
MP SCAN: 05:56:58 Применяется правило 4: O -> aO'
MP SCAN: 05:56:58 Промежуточное представление: aO'S'
MP SCAN: 05:56:58 Применяется правило 8: O' -> =R
MP SCAN: 05:56:58 Промежуточное представление: a=RS'
MP SCAN: 05:56:58 Применяется правило 10: R -> (EE'S'
MP SCAN: 05:56:58 Промежуточное представление: a=(EE'S'
MP SCAN: 05:56:58 Применяется правило 19: E -> a
MP SCAN: 05:56:58 Промежуточное представление: a=(aE'S'
MP SCAN: 05:56:58 Применяется правило 22: E' -> +E)
MP SCAN: 05:56:58 Промежуточное представление: a=(a+E)S'
MP SCAN: 05:56:58 Ошибка
MP SCAN: 05:56:58 Stack: E)S'Δ
MP SCAN: 05:56:58 Оставшаяся часть строки: -\
error during analyze. stack symbol <E> and string symbol <->
```

6.

```
Ввод: {a<a}
MP SCAN: 05:57:13 Промежуточное представление: S
MP SCAN: 05:57:13 Применяется правило 1: S -> OS'
MP SCAN: 05:57:13 Промежуточное представление: OS'
MP SCAN: 05:57:13 Применяется правило 5: O -> {Y[S]}
MP SCAN: 05:57:13 Промежуточное представление: {Y[S]}S'
MP SCAN: 05:57:13 Применяется правило 15: Y -> aY'
MP SCAN: 05:57:13 Промежуточное представление: {aY'[S]}S'
MP SCAN: 05:57:13 Применяется правило 17: Y' -> <a
MP SCAN: 05:57:13 Промежуточное представление: {a<a[S]}S'
MP SCAN: 05:57:13 Ошибка
MP SCAN: 05:57:13 Stack: [S]}S'Δ
MP SCAN: 05:57:13 Оставшаяся часть строки: ]-\
error during analyze. stack symbol <[> and string symbol <]>
```

Все результаты совпали с ожидаемыми и полученными предыдущей программой.

Вывод:

Мы изучили и научились применять нисходящие методы обработки формальных языков, реализовав распознаватель методом рекурсивного спуска и МП-распознаватель.